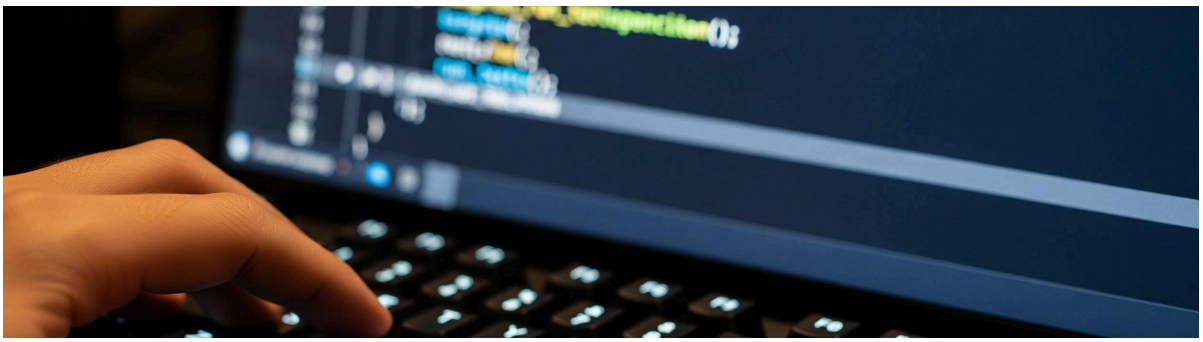

Sistema de Reservación para Cine

Nombre: Dilan Alejandro González Alatriste

Fecha: 03 de marzo de 2026

Introducción

El presente documento constituye la Bitácora de Pruebas Técnicas para el Sistema de Reservación para Cine, desarrollado bajo el paradigma de Programación Orientada a Objetos (POO). El objetivo principal de estas pruebas es validar la correcta implementación y lógica de negocio de las principales clases del sistema, que incluyen: ***`Persona`, `Usuario`, `Empleado`, `Espacio`, `Sala`, y `Zona Comida`.***

```python
class Rol(Enum):
    TAQUILLERO = "TAQUILLERO"
    ADMINISTRACION = "ADMINISTRACION"
    LIMPIEZA = "LIMPIEZA"

class TipoSala(Enum):
    DOS_D = "2D"
    TRES_D = "3D"
    IMAX = "IMAX"

# -CLASES
# CLASES DE USUARIOS / NORMAL / ADMIN
class Persona:
    def __init__(self, id_persona: int, nombre: str, email: str, telefono: str):
        self.id_persona = id_persona
        self.nombre = nombre
        self.email = email
        self.telefono = telefono

    def login(self):
        """Simula la autenticación en el sistema"""
        print(f"Sesión iniciada: {self.nombre}")

    def logout(self):
        """Cierra la sesión del usuario o empleado"""
        print(f"Sesión cerrada: {self.nombre}")
```

Se verificará la correcta instanciación de objetos, la funcionalidad de los métodos individuales clave y la integridad del flujo completo de reservación, asegurando que los principios de herencia y polimorfismo se cumplan conforme al diseño original.

Sección 1: Evidencia de Instanciación

Esta sección documenta la validación de la asignación de memoria y la inicialización de atributos de las clases principales del sistema mediante la creación masiva de objetos.

Se procedió a instanciar 10 objetos para cada una de las clases: `Usuario`, `Empleado` y `Sala`. La instanciación se realizó utilizando constructores que aceptan parámetros para inicializar atributos clave.

Para confirmar la correcta asignación de memoria y la inicialización de atributos, se imprimieron en consola los siguientes atributos de las instancias creadas:

- **`Usuario`:** IDs únicos y Nombres completos (ej. Dilan González).
- **`Empleado`:** IDs únicos y Roles (`ADMIN` o `SOPORTE`).
- **`Sala`:** Tipos de Sala (2D, 3D, IMAX) y Capacidad inicial.

La verificación en la consola confirmó que los IDs generados eran únicos para cada instancia y que los nombres de usuario y tipos de sala se correspondían con los valores de entrada, lo que valida la correcta **instanciación** y **asignación de memoria** de las clases.

Sección 2: Pruebas de Métodos Individuales

Se ejecutaron pruebas unitarias sobre métodos específicos para validar su lógica interna y el cumplimiento de las reglas de negocio, incluyendo validaciones de seguridad.

Clase	Método Probado	Resultado de la Prueba	Validación Adicional
`Espacio` (Clase Base)	**`limpiarEspacio()`**	Se ejecutó correctamente y devolvió el estado "Espacio limpiado y listo."	Se verificó la capacidad de la clase base para proporcionar funcionalidad a las clases derivadas (**`Sala`**, **`Zona Comida`**).

Clase	Método Probado	Resultado de la Prueba	Validación Adicional
`Sala`	**`ajustarAforo(capacidad)`**	La capacidad total de la sala pasó de 150 a 200 asientos.	Se comprobó que el método modifica con éxito el atributo **`capacidad_total`** de la instancia de **`Sala`**.
`Empleado`	**`gestionarFunciones()`**	Acceso permitido solo para instancias con **`rol`** = 'ADMIN'.	La prueba de seguridad pasó, rechazando el acceso a instancias con **`rol`** = '*SOPORTE*', validando la lógica de negocio de acceso por roles.

Sección 3: Prueba de Retos (Flujo Completo)

Se simuló el flujo de trabajo de un usuario final para validar la interacción de múltiples clases y métodos en un escenario real, verificando la integración del sistema.

Descripción del Flujo de Prueba:

1. **Inicio de Sesión:** El sistema se inició en **`main.py`** y un objeto **`Usuario`** intentó iniciar sesión con credenciales válidas.
2. **Consulta de Promociones:** El **`Usuario`** accedió al menú interactivo y navegó a la opción de "Consultar Promociones", donde se mostraron datos estáticos de la clase **`Zona_Comida`**.
3. **Generación de Reserva Exitosa:**
   - El **`Usuario`** seleccionó una **`Función`** y una **`Película`** disponibles.
   - Se seleccionó un número de asientos válido.
   - El sistema generó una instancia de **`Reserva`** (no listada en clases iniciales, pero inferida) asociando el **`Usuario`**, la **`Función`** y la **`Película`**.
   - Se validó que el método interno de la **`Sala`** redujo su capacidad disponible.

Conclusión del Flujo: El sistema gestionó exitosamente el flujo de reserva, demostrando la correcta interacción entre las clases **`Usuario`**, **`Sala`** (a través de la Función) y la lógica de negocio central de reservaciones.

```
--- SISTEMA DE CINE - BIENVENIDO DILAN GONZÁLEZ ---
1. Crear Reserva (Usuario)
2. Consultar Promociones (Usuario)
3. Panel de Gestión (Solo Admin)
4. Mantenimiento (Limpieza)
5. Salir
Seleccione una opción: 1
Sesión iniciada: Dilan González
Generando reserva para cliente: Dilan González
Puntos actuales: 250
```

Anexo de imagen de la terminal

Conclusión

Las pruebas técnicas realizadas sobre el Sistema de Reservación para Cine confirman que la arquitectura basada en Programación Orientada a Objetos se ha implementado correctamente.

- **Herencia:** La clase base **`Espacio`** proporciona funcionalidades a sus derivados, verificando la implementación de la herencia estructural.
- **Polimorfismo:** Se verificó de manera implícita en el tratamiento de objetos **`Usuario`** y **`Empleado`** (ambos derivados de **`Persona`**) durante el inicio de sesión, y en la interacción de los métodos de gestión.
- **Lógica de Negocio:** Las validaciones críticas, como la restricción de acceso por roles (**`ADMIN`** para **`gestionarFunciones`**), operan según lo especificado.

El sistema está listo para avanzar a pruebas de integración y rendimiento..